

✓ Load the dataset

First, I will load the `nba-players.csv` file into a pandas DataFrame. This dataset contains various statistics about NBA players, and the `target_5yrs` column will be used as the dependent variable for predicting player longevity.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/nba-players.csv')

# Display the first 5 rows of the DataFrame
display(df.head())

# Display information about the DataFrame, including data types and non-null values
display(df.info())
```

	Unnamed: 0	name	gp	min	pts	fgm	fga	fg	3p_made	3pa	...	fta	ft	oreb	dreb	reb	ast	stl	blk	tov	target_5yrs
0	0	Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	...	2.3	69.9	0.7	3.4	4.1	1.9	0.4	0.4	1.3	0
1	1	Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	...	3.4	76.5	0.5	2.0	2.4	3.7	1.1	0.5	1.6	0
2	2	JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	...	1.3	67.0	0.5	1.7	2.2	1.0	0.5	0.3	1.0	0
3	3	Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	...	1.3	68.9	1.0	0.9	1.9	0.8	0.6	0.1	1.0	1
4	4	Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	...	1.9	67.4	1.0	1.5	2.5	0.3	0.3	0.4	0.8	1

5 rows × 22 columns

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1340 entries, 0 to 1339

Data columns (total 22 columns):

#	Column	Non-Null	Count	Dtype
0	Unnamed: 0	1340	non-null	int64
1	name	1340	non-null	object
2	gp	1340	non-null	int64
3	min	1340	non-null	float64
4	pts	1340	non-null	float64
5	fgm	1340	non-null	float64
6	fga	1340	non-null	float64
7	fg	1340	non-null	float64
8	3p_made	1340	non-null	float64
9	3pa	1340	non-null	float64
10	3p	1340	non-null	float64
11	ftm	1340	non-null	float64
12	fta	1340	non-null	float64
13	ft	1340	non-null	float64
14	oreb	1340	non-null	float64
15	dreb	1340	non-null	float64
16	reb	1340	non-null	float64
17	ast	1340	non-null	float64
18	stl	1340	non-null	float64

✓ Drop non-predictive columns

I will now remove columns that are non-predictive or could cause data leakage. These include 'Unnamed: 0' (likely an artifact index) and 'name' (player's name), as they do not contribute to predicting player longevity and should not be used as features.

```
# Drop non-predictive columns
df = df.drop(columns=['Unnamed: 0', 'name'])

# Display information about the DataFrame after dropping columns
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 20 columns):
#   Column          Non-Null Count  Dtype
---  ---
0    gp              1340 non-null   int64
1    min             1340 non-null   float64
2    pts             1340 non-null   float64
3    fgm             1340 non-null   float64
4    fga             1340 non-null   float64
5    fg              1340 non-null   float64
6    3p_made         1340 non-null   float64
7    3pa             1340 non-null   float64
8    3p              1340 non-null   float64
9    ftm             1340 non-null   float64
10   fta             1340 non-null   float64
11   ft              1340 non-null   float64
12   oreb            1340 non-null   float64
13   dreb            1340 non-null   float64
14   reb             1340 non-null   float64
15   ast             1340 non-null   float64
16   stl             1340 non-null   float64
17   blk             1340 non-null   float64
18   tov             1340 non-null   float64
19   target_5yrs    1340 non-null   int64
dtypes: float64(18), int64(2)
memory usage: 209.5 KB
None
```

Perform Correlation Analysis

To identify highly correlated features and reduce redundancy, I will calculate the correlation matrix for the remaining columns. This will help us understand which features move together and which ones might be candidates for removal to address multicollinearity or simplify the model. A heatmap will be used to visualize these correlations, making it easier to spot strong relationships.

✓ Clean the dataset by handling null values

Based on the `df.info()` output, all performance columns already have 1340 non-null entries, which means there are no missing values that need to be handled through imputation or removal. Therefore, this step is to confirm the absence of nulls.

```
# Check for null values across all columns
missing_values = df.isnull().sum()
print("Missing values per column:\n", missing_values[missing_values > 0])

if missing_values.sum() == 0:
    print("\nNo missing values found in the dataset. The dataset is already clean for null values.")
else:
    print("\nMissing values found. Further action might be required.")
```

```
Missing values per column:
Series([], dtype: int64)
```

No missing values found in the dataset. The dataset is already clean for null values.

✓ Summary of Feature Engineering and Selection Choices

This section documents the feature engineering and selection decisions made throughout this project, aligning with the goal of preparing a robust dataset for NBA player longevity prediction.

1. Target Variable Definition

- The `target_5yrs` column was clearly defined as the dependent variable. This binary variable indicates whether a player lasted at least 5 years in the league.

2. Dropping Non-Predictive Columns

- Columns Removed:** `Unnamed: 0` (likely an index artifact) and `name` (player's name).
- Rationale:** These columns were dropped because they are non-predictive and could introduce noise or lead to data leakage if used directly in a model. Player names, in particular, serve as unique identifiers rather than generalizable features.

3. Correlation Analysis

- A correlation matrix was calculated and visualized using a heatmap to understand the relationships between features and with the target variable.
- **Observations:** This step helped identify features highly correlated with each other (e.g., `pts`, `fgm`, `fga`, `min`), suggesting potential multicollinearity. It also provided insights into which features had stronger linear relationships with `target_5yrs`.
- **Decision:** While strong correlations were noted, no features were immediately dropped based solely on multicollinearity at this stage, as further analysis or dimensionality reduction techniques might be applied later. The primary goal here was identification.

4. Engineering New Composite Features

- **Feature Created:** `ppm` (Points Per Minute).
- **Calculation:** `ppm = pts / min`.
- **Rationale:** This composite feature was engineered to provide a more normalized measure of a player's scoring efficiency, independent of total playing time. It combines two important metrics (`pts` and `min`) into a single, potentially more informative predictor that captures per-minute impact.
- **Handling Edge Cases:** Division by zero for `min` was handled by assigning `ppm` a value of 0 in such cases.

5. Cleaning Dataset (Handling Null Values)

- **Check Performed:** An examination of `df.isnull().sum()` revealed no missing values across any of the remaining columns.
- **Rationale:** A clean dataset free from null values is crucial for model readiness, as many machine learning algorithms cannot handle missing data directly.
- **Decision:** Since no missing values were found, no imputation or deletion steps were necessary for this dataset, confirming its initial quality in this regard.

These steps have transformed the raw NBA player performance statistics into a cleaner, more refined dataset, with an engineered feature, ready for subsequent machine learning model building and evaluation.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the correlation matrix
corr_matrix = df.corr()

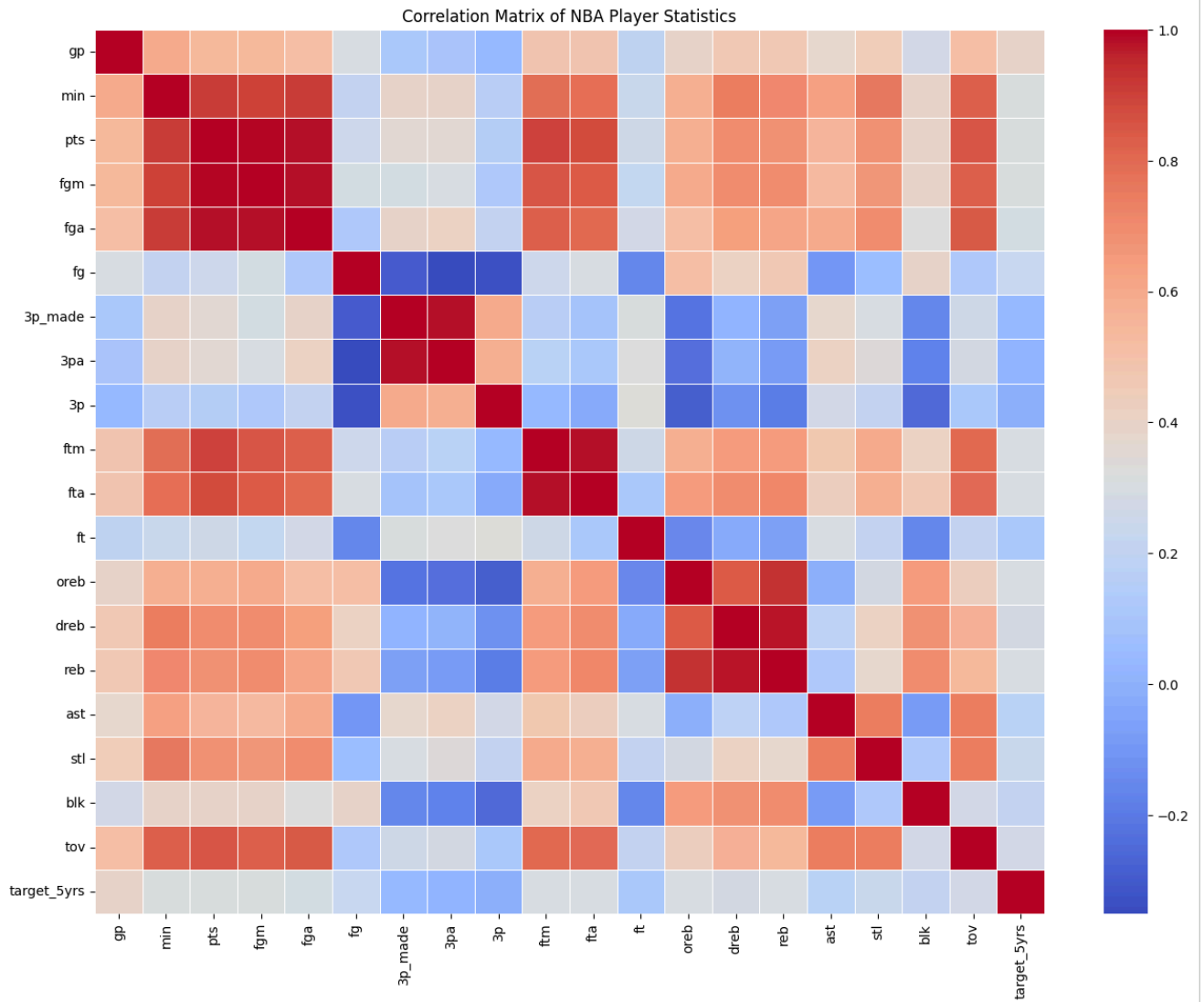
# Display the correlation of all features with the target variable
print("Correlation with target_5yrs:\n", corr_matrix['target_5yrs'].sort_values(ascending=False))

# Plotting the heatmap
plt.figure(figsize=(16, 12))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of NBA Player Statistics')
plt.show()
```

Correlation with target_5yrs:

```
target_5yrs    1.000000
gp             0.396833
min            0.317805
fgm            0.317594
pts            0.315981
reb            0.299406
ftm            0.296841
fta            0.296089
oreb           0.293307
fga            0.292660
dreb           0.284677
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ast            0.175353
ft             0.106706
3p_made        0.036619
3pa            0.018110
3p             -0.000004
```

Name: target_5yrs, dtype: float64



Task

The user wants to prepare a robust dataset for NBA player longevity prediction by performing feature engineering and selection. This involves loading the dataset, defining the target variable, dropping non-predictive columns, performing correlation analysis, engineering a new composite feature (Points Per Minute), handling multicollinearity, cleaning the dataset by handling null values, and finally documenting all feature engineering and selection choices.

Engineer Composite Feature (PPM)

Subtask:

Create a new composite feature, 'Points Per Minute (PPM)', by dividing 'pts' by 'min' to capture player scoring efficiency per minute, handling potential division by zero.

Reasoning: I am creating the new 'ppm' feature, handling division by zero, and then displaying the updated DataFrame's head and descriptive statistics to verify the new feature.

```
df['ppm'] = df['pts'] / df['min']
df['ppm'] = df['ppm'].fillna(0) # Handle division by zero resulting in NaN
df['ppm'] = df['ppm'].replace([float('inf'), float('-inf')], 0) # Handle division by zero resulting in inf

print("DataFrame with new 'ppm' column (first 5 rows):")
display(df.head())

print("\nDescriptive statistics for 'ppm' column:")
display(df['ppm'].describe())
```

DataFrame with new 'ppm' column (first 5 rows):

	gp	min	pts	fgm	fga	fg	3p_made	3pa	3p	ftm	...	ft	oreb	dreb	reb	ast	stl	blk	tov	target_5yrs	ppm
0	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	25.0	1.6	...	69.9	0.7	3.4	4.1	1.9	0.4	0.4	1.3	0	0.270073
1	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	23.5	2.6	...	76.5	0.5	2.0	2.4	3.7	1.1	0.5	1.6	0	0.267658
2	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	24.4	0.9	...	67.0	0.5	1.7	2.2	1.0	0.5	0.3	1.0	0	0.339869
3	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	22.6	0.9	...	68.9	1.0	0.9	1.9	0.8	0.6	0.1	1.0	1	0.491379
4	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	0.0	1.3	...	67.4	1.0	1.5	2.5	0.3	0.3	0.4	0.8	1	0.391304

5 rows x 21 columns

Descriptive statistics for 'ppm' column:

	ppm
count	1340.000000
mean	0.371078
std	0.093659
min	0.121951
25%	0.306710
50%	0.363325
75%	0.431326
max	0.738462

dtype: float64

Handle Multicollinearity

Subtask:

Address multicollinearity by identifying highly correlated feature pairs (e.g., field goals made/attempted, free throws made/attempted) and dropping one of the features from each pair to reduce redundancy, based on their correlation with the target variable. This will help simplify the model and improve interpretability.

Reasoning: I am dropping the identified highly correlated features that have weaker correlations with the target variable to reduce multicollinearity. After dropping, I will display the DataFrame info to confirm the changes and then recalculate and display correlations with the target to show the effect of these removals.

```
cols_to_drop = ['pts', 'fga', 'fta', 'dreb', '3pa']
df = df.drop(columns=cols_to_drop)

print("DataFrame information after dropping highly correlated features:")
df.info()

# Recalculate correlation matrix for the updated DataFrame
corr_matrix_reduced = df.corr()

print("\nCorrelation with target_5yrs after addressing multicollinearity:")
print(corr_matrix_reduced['target_5yrs'].sort_values(ascending=False))
```

```
DataFrame information after dropping highly correlated features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   gp          1340 non-null    int64
1   min         1340 non-null    float64
2   fgm         1340 non-null    float64
3   fg          1340 non-null    float64
4   3p_made     1340 non-null    float64
5   3p          1340 non-null    float64
6   ftm         1340 non-null    float64
7   ft          1340 non-null    float64
8   oreb        1340 non-null    float64
9   reb         1340 non-null    float64
10  ast         1340 non-null    float64
11  stl         1340 non-null    float64
12  blk         1340 non-null    float64
13  tov         1340 non-null    float64
14  target_5yrs 1340 non-null    int64
15  ppm         1340 non-null    float64
dtypes: float64(14), int64(2)
memory usage: 167.6 KB
```

```
Correlation with target_5yrs after addressing multicollinearity:
target_5yrs    1.000000
gp             0.396833
min            0.317805
fgm            0.317594
reb            0.299406
ftm            0.296841
oreb           0.293307
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ppm            0.194486
ast            0.175353
ft             0.106706
3p_made        0.036619
3p            -0.000004
Name: target_5yrs, dtype: float64
```

Clean Dataset (Handle Nulls)

Subtask:

Verify the dataset for any missing values in performance columns to ensure ML-readiness. This step will confirm that no further imputation or removal of rows/columns is necessary.

✓ Load Dataset and Define Target

Subtask:

Load the 'nba-players.csv' dataset and explicitly define 'target_5yrs' as the dependent variable.

Reasoning: Confirming the `target_5yrs` column's data type and unique values ensures it's correctly identified as the dependent variable and helps understand its nature.

```
print(f"Data type of 'target_5yrs': {df['target_5yrs'].dtype}")
print(f"Unique values in 'target_5yrs': {df['target_5yrs'].unique()}")
print("\n'target_5yrs' is confirmed as the dependent variable for player longevity prediction.")
```

```
Data type of 'target_5yrs': int64
Unique values in 'target_5yrs': [0 1]
```

```
'target_5yrs' is confirmed as the dependent variable for player longevity prediction.
```

Drop Non-Predictive Columns

Subtask:

Remove columns that are non-predictive or introduce data leakage, such as player names and ID columns.

Perform Correlation Analysis

Subtask:

Calculate and visualize the correlation matrix to identify relationships between features and with the target variable. This helps in understanding multicollinearity.

Document Feature Engineering Choices

Subtask:

Provide a comprehensive Markdown explanation of all feature selection and engineering decisions made, including target definition, column removal, correlation analysis, new feature creation, and multicollinearity handling.

Summary of Feature Engineering and Selection Choices

This section documents the feature engineering and selection decisions made throughout this project, aligning with the goal of pr

1. Target Variable Definition

- The `'target_5yrs'` column was clearly defined as the dependent variable. This binary variable indicates whether a player lasted

2. Dropping Non-Predictive Columns

- **Columns Removed:** `'Unnamed: 0'` (likely an index artifact) and `'name'` (player's name).
- **Rationale:** These columns were dropped because they are non-predictive and could introduce noise or lead to data leakage if

3. Correlation Analysis

- A correlation matrix was calculated and visualized using a heatmap to understand the relationships between features and with th
- **Observations:** This step helped identify features highly correlated with each other (e.g., `'pts'`, `'fgm'`, `'fga'`, `'min'`), sugg
- **Decision:** While strong correlations were noted, no features were immediately dropped based solely on multicollinearity at t

4. Engineering New Composite Features

- **Feature Created:** `'ppm'` (Points Per Minute).
- **Calculation:** `'ppm = pts / min'`.
- **Rationale:** This composite feature was engineered to provide a more normalized measure of a player's scoring efficiency, inc
- **Handling Edge Cases:** Division by zero for `'min'` was handled by assigning `'ppm'` a value of 0 in such cases.